yolov8n目标检测

yolov8n**目标检测**
例程实验效果
代码讲解
代码结构
代码解析
算法简述
什么是yolo?

例程实验效果

本节我们来学习一下使用K230实现基于yolov8的目标检测

本节我们使用的模型支持识别近80种物品，具体支持识别的物品类型我们可以参考 代码中 labels 标签数组的内容

简单整理一下有如下内容

```
交通工具/Transportation：
自行车/bicycle，汽车/car，摩托车/motorcycle，飞机/airplane，公共汽车/bus，火车/train，卡车/truck，船/boat

交通设施/Traffic Facilities：
交通信号灯/traffic light，消防栓/fire hydrant，停车标志/stop sign，停车计时器/parking meter

动物/Animals：
鸟/bird，猫/cat，狗/dog，马/horse，羊/sheep，牛/cow，大象/elephant，熊/bear，斑马/zebra，长颈鹿/giraffe

个人物品/Personal Items：
背包/backpack，雨伞/umbrella，手提包/handbag，领带/tie，行李箱/suitcase

运动用品/Sports Equipment：
飞盘/frisbee，滑雪板/skis，单板滑雪/snowboard，运动球/sports ball，风筝/kite，棒球棒/baseball bat，棒球手套/baseball glove，滑板/skateboard，冲浪板/surfboard，网球拍/tennis racket

餐饮用品/Dining Items：
瓶子/bottle，酒杯/wine glass，杯子/cup，叉子/fork，刀/knife，勺子/spoon，碗/bowl

食物/Food：
香蕉/banana，苹果/apple，三明治/sandwich，橙子/orange，西兰花/broccoli，胡萝卜/carrot，热狗/hot dog，披萨/pizza，甜甜圈/donut，蛋糕/cake

家居用品/Home Items：
长凳/bench，椅子/chair，沙发/couch，盆栽/potted plant，床/bed，餐桌/dining table，马桶/toilet，花瓶/vase

电子设备/Electronics：
```

```
电视/tv，笔记本电脑/laptop，鼠标/mouse，遥控器/remote，键盘/keyboard，手机/cell phone，微波炉/microwave，烤箱/oven，烤面包机/toaster，水槽/sink，冰箱/refrigerator，吹风机/hair drier

其他/Others：
书/book，时钟/clock，剪刀/scissors，泰迪熊/teddy bear，牙刷/toothbrush
```

连接IDE后运行本节例程代码，然后我们找几个简单的场景来实验一下



可以看到K230能正确的识别到我们这些物品，并给出打分

这个模型能识别的东西非常多，在这里我就不一一例举了，大家可以根据上面我列出来的表格自行尝试

> 当前例程已添加串口输出
>
> 协议格式请参考资料中的【例程通信协议.xlsx】

代码讲解

代码结构

1. 初始化阶段/Initialization Phase:
    - 加载YOLOv8模型/Load YOLOv8 model
    - 加载标签/Load labels
    - 设置参数/Set parameters
    - 初始化AI2D/Initialize AI2D
    - 生成类别颜色/Generate class colors
2. 预处理流程/Preprocessing Flow:
    - 配置预处理/Configure preprocessing
    - 计算填充/Calculate padding
    - 缩放图像/Resize image
    - 归一化/Normalize
3. 检测流程/Detection Flow:
    - 运行模型/Run model
    - 转置输出/Transpose output
    - 后处理/Postprocess

    - 非极大值抑制/NMS
4. 绘制效果流程/Drawing Flow:
    - 清空显示/Clear display
    - 绘制检测框/Draw boxes
    - 绘制标签/Draw labels
    - 更新显示/Update display
5. 主循环/Main Loop:
    - 获取帧/Get frame
    - 处理流程/Process flow
    - 垃圾回收/Garbage collection
6. 退出流程/Exit Flow:
    - 退出演示/Exit demo
    - 清理资源/Clean up

代码解析

完整代码请参考文件【源码汇总 / 09.Scene / 03.object_detect_yolov8n】

```
# 自定义YOLOv8检测类 Custom YOLOv8 detection class
class ObjectDetectionApp(AIBase):
    def
__init__(self,kmodel_path,labels,model_input_size,max_boxes_num,confidence_thres
hold=0.5,nms_threshold=0.2,rgb888p_size=[224,224],display_size=
[1920,1080],debug_mode=0):
        """
        初始化函数 Initialization function
        kmodel_path: 模型路径 Model path
        labels: 标签列表 Label list
        model_input_size: 模型输入大小 Model input size
        max_boxes_num: 最大检测框数量 Maximum number of detection boxes
        confidence_threshold: 置信度阈值 Confidence threshold
        nms_threshold: 非极大值抑制阈值 Non-maximum suppression threshold
        rgb888p_size: RGB图像大小 RGB image size
        display_size: 显示大小 Display size
        debug_mode: 调试模式 Debug mode
        """
        super().__init__(kmodel_path,model_input_size,rgb888p_size,debug_mode)
        self.kmodel_path = kmodel_path
        self.labels = labels
        # 模型输入分辨率 Model input resolution
        self.model_input_size = model_input_size
        # 阈值设置 Threshold settings
        self.confidence_threshold = confidence_threshold
        self.nms_threshold = nms_threshold
        self.max_boxes_num = max_boxes_num
        # sensor给到AI的图像分辨率 Image resolution from sensor to AI
        self.rgb888p_size = [ALIGN_UP(rgb888p_size[0],16),rgb888p_size[1]]
        # 显示分辨率 Display resolution
        self.display_size = [ALIGN_UP(display_size[0],16),display_size[1]]
        self.debug_mode = debug_mode
        # 检测框预置颜色值 Preset colors for detection boxes
        self.color_four = get_colors(len(self.labels))
        # 宽高缩放比例 Width and height scaling ratios
        self.x_factor = float(self.rgb888p_size[0])/self.model_input_size[0]
        self.y_factor = float(self.rgb888p_size[1])/self.model_input_size[1]
```

```
        # Ai2d实例，用于实现模型预处理 Ai2d instance for model preprocessing
        self.ai2d = Ai2d(debug_mode)
        # 设置Ai2d的输入输出格式和类型 Set Ai2d input/output format and type

 self.ai2d.set_ai2d_dtype(nn.ai2d_format.NCHW_FMT,nn.ai2d_format.NCHW_FMT,np.uin
t8, np.uint8)

    # 配置预处理操作 Configure preprocessing operations
    def config_preprocess(self,input_image_size=None):
        """
        配置图像预处理参数 Configure image preprocessing parameters
        input_image_size: 输入图像大小(可选) Input image size (optional)
        """
        with ScopedTiming("set preprocess config",self.debug_mode > 0):
            # 初始化ai2d预处理配置 Initialize ai2d preprocessing configuration
            ai2d_input_size = input_image_size if input_image_size else
self.rgb888p_size
            top,bottom,left,right,self.scale =
letterbox_pad_param(self.rgb888p_size,self.model_input_size)
            # 配置padding预处理 Configure padding preprocessing
            self.ai2d.pad([0,0,0,0,top,bottom,left,right], 0, [128,128,128])
            self.ai2d.resize(nn.interp_method.tf_bilinear,
nn.interp_mode.half_pixel)
            self.ai2d.build([1,3,ai2d_input_size[1],ai2d_input_size[0]],
[1,3,self.model_input_size[1],self.model_input_size[0]])

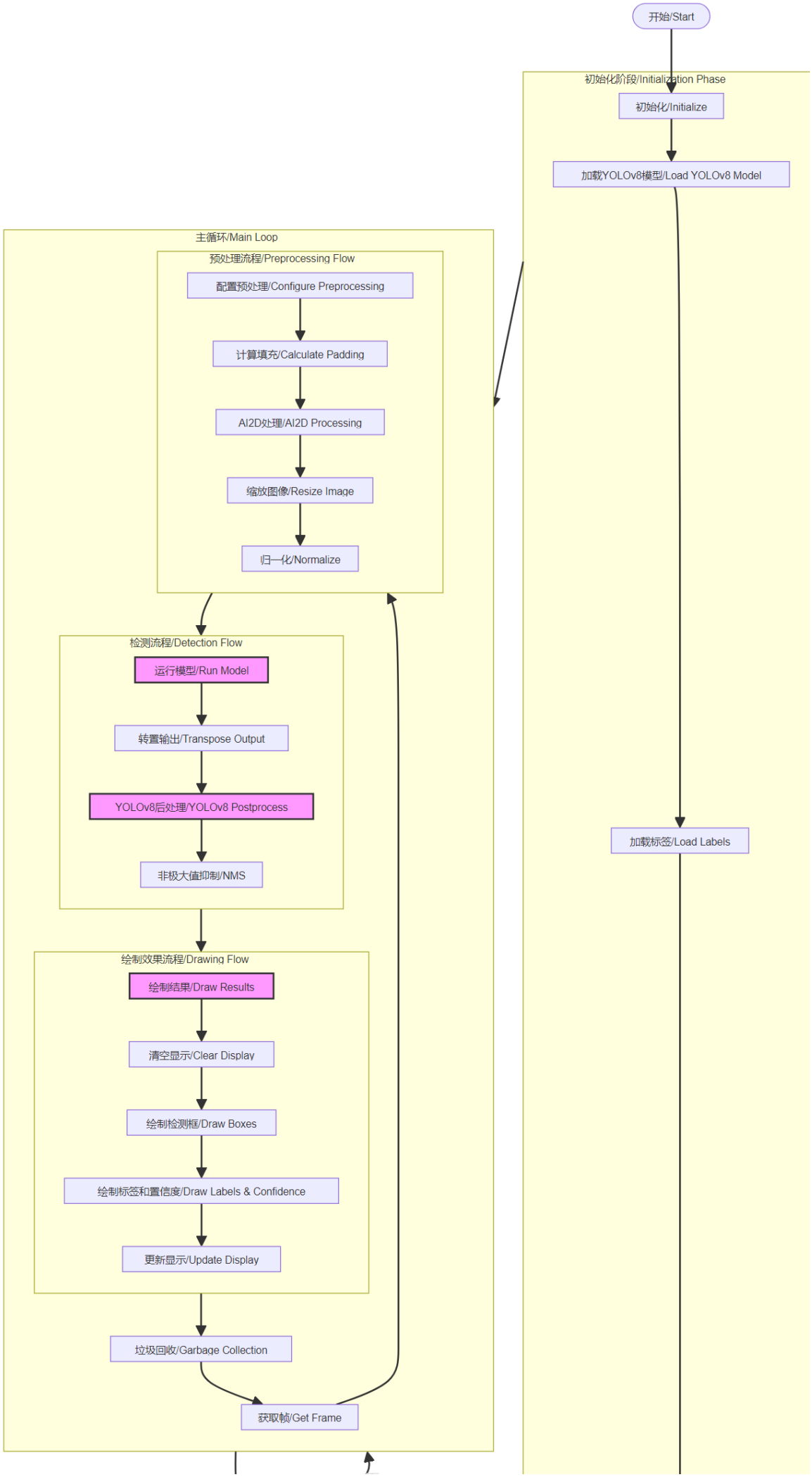
    # 预处理函数 Preprocessing function
    def preprocess(self,input_np):
        with ScopedTiming("preprocess",self.debug_mode > 0):
            return [nn.from_numpy(input_np)]

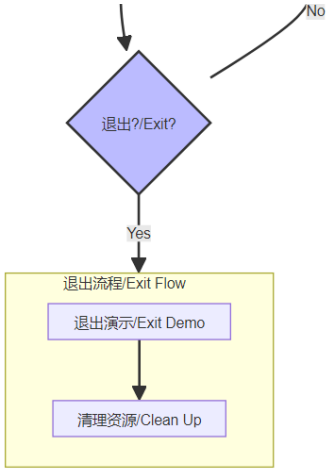
    # 后处理函数 Postprocessing function
    def postprocess(self,results):
        """
        处理模型输出结果 Process model output results
        results: 模型输出 Model output
        """
        with ScopedTiming("postprocess",self.debug_mode > 0):
            new_result = results[0][0].transpose()
            det_res = aidemo.yolov8_det_postprocess(new_result.copy(),
                [self.rgb888p_size[1],self.rgb888p_size[0]],
                [self.model_input_size[1],self.model_input_size[0]],
                [self.display_size[1],self.display_size[0]],
                len(self.labels),
                self.confidence_threshold,
                self.nms_threshold,
                self.max_boxes_num)
            return det_res

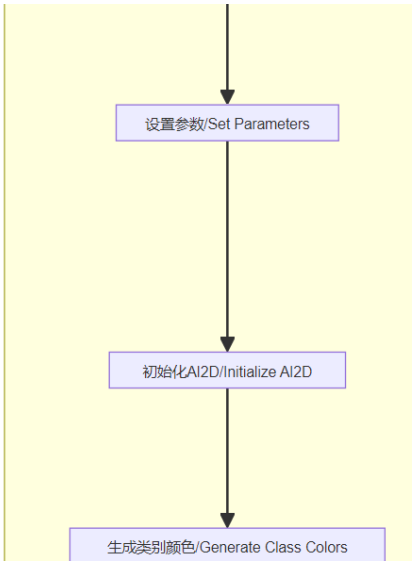
    # 绘制结果函数 Draw results function
    def draw_result(self,pl,dets):
        """
        绘制检测结果 Draw detection results
        pl: PipeLine实例 PipeLine instance
        dets: 检测结果 Detection results
        """
        with ScopedTiming("display_draw",self.debug_mode >0):
            if dets:
```

```
                pl.osd_img.clear()
                for i in range(len(dets[0])):
                    x, y, w, h = map(lambda x: int(round(x, 0)), dets[0][i])
                    # 绘制矩形框和标签 Draw rectangle box and label
                    pl.osd_img.draw_rectangle(x,y, w, h,
color=self.color_four[dets[1][i]],thickness=4)
                    pl.osd_img.draw_string_advanced( x , y-50,32,
                        " " + self.labels[dets[1][i]] + " " + str(round(dets[2]
[i],2)) ,
                        color=self.color_four[dets[1][i]])
            else:
                pl.osd_img.clear()
```

流程图如下：


开始/Start
初始化阶段/Initialization Phase
初始化/Initialize
加载YOLOv8模型/Load YOLOv8 Model
加载标签/Load Labels
主循环/Main Loop
预处理流程/Preprocessing Flow
配置预处理/Configure Preprocessing
计算填充/Calculate Padding
AI2D处理/AI2D Processing
缩放图像/Resize Image
归一化/Normalize
检测流程/Detection Flow
运行模型/Run Model
转置输出/Transpose Output
YOLOv8后处理/YOLOv8 Postprocess
非极大值抑制/NMS
绘制效果流程/Drawing Flow
绘制结果/Draw Results
清空显示/Clear Display
绘制检测框/Draw Boxes
绘制标签和置信度/Draw Labels & Confidence
更新显示/Update Display
垃圾回收/Garbage Collection
获取帧/Get Frame





算法简述

什么是yolo?

YOLO (You Only Look Once) 是一个广受欢迎的目标检测算法系列。以下是主要特点：

YOLO基本原理：

- 将图像分成网格
- 直接预测边界框和类别概率
- 单阶段检测，速度快

YOLOv8n特点（n表示nano，最小版本）：

- 更快的推理速度
- 更小的模型体积 (~3MB)
- 使用C2f模块和SPPF结构
- 适合移动设备部署

主要性能对比：
YOLOv8n在COCO数据集上：

- AP: 37.3%
- 速度: 比v5快约30%
- 参数量: 3.2M